Tightening the RL–Optimal-Control Unification in PITS-MRAS: A Mathematical and Architectural Blueprint

TL;DR

- The single highest-leverage move is to recognize your MRAS Lyapunov function $V(e)=e^{\top}Pe$ **is** an LQR/tracking value function: replacing the manual `V̇<0 → emergency backup` heuristic with (a) an Integral-RL policy-evaluation update (Vrabie–Lewis 2009) and (b) a CLF-CBF-QP safety filter (Ames et al. 2017) gives you formal optimality + forward-invariance guarantees for near-zero implementation cost — these two are your best "bang for buck."
- Six of the ten connections are already rigorous, publishable identities (Lyapunov-equation = ARE special case via Kleinman's 1968 algorithm; H_θ as storage/value function; costate $\lambda=\nabla V$=critic gradient; MRAS law as deterministic policy gradient; SAC-$\alpha = R^{-1}$ control penalty; $H\infty$ = two-player GARE game). The remaining four (physics-informed Bellman/HJB residual, TD-MPC2 latent planning, neural Lyapunov certificates, generalized Lyapunov) are sound but add the most implementation complexity.
- Concrete deliverables below: ~7 new loss terms, 3 new network heads (critic/value, disturbance/adversary, CBF filter), and a re-derived adaptation law that unifies your MRAS gradient with integral-RL and actor-critic updates.

Key Findings

1. Your Lyapunov function is literally a value function (ADP / integral RL). For the linear reference model $\dot{x}_m = A_m x_m + B_m r$ with quadratic $V(e)=e^{\top}Pe$, the Lyapunov equation $A_m^{\top}P + PA_m = -Q$ is exactly the policy-evaluation step of Kleinman's algorithm (D. L. Kleinman, "On an Iterative Technique for Riccati Equation Computations," IEEE Trans. Automatic Control AC-13(1):114–115, Feb. 1968, DOI 10.1109/TAC.1968.1098829), SCIRP which is continuous-time policy iteration on the algebraic Riccati equation. Vrabie & Lewis ("Neural network approach to continuous-time direct adaptive optimal control for partially unknown nonlinear systems," Neural Networks 22(3):237–246, Apr. 2009, DOI 10.1016/j.neunet.2009.03.008) showed this can be run *model-free* via the Integral Reinforcement Learning (IRL) Bellman equation. ResearchGate Wikidata This is the deepest

and most actionable connection.

2. The port-Hamiltonian Hamiltonian H_θ is a storage/value function. IDA-PBC energy shaping and RL value functions are the same object viewed from passivity vs. optimality; H_d (desired Hamiltonian) plays the role of the value function, and L2-gain/H∞ performance is a passivity (dissipation-inequality) statement.

3. Costate = critic gradient = ∇H. PMP's $\lambda(t)=\partial V/\partial x$ is the same object as the critic's value gradient and, in the port-Hamiltonian decoder, the generalized force $\nabla_q H$ / velocity $\nabla_p H$. This justifies an architecture where the attention context c_t is trained to approximate ∇V.

4. MRAS law ↔ deterministic policy gradient. Both are gradient steps that chain a regressor/Jacobian against a value-gradient term.

5. SAC entropy temperature α ≡ control-penalty weight R^{-1}. Maximum-entropy RL with quadratic reward recovers LQR/Gaussian policies whose covariance scales with R; α and R play dual roles.

6. CLF-CBF-QP is the rigorous safety filter. Your V(e) doubles as a CLF; a CBF h(x) for the tracking-error system gives a single-constraint QP with a closed-form solution.

7. H∞ robust PITS-MRAS = two-player zero-sum game. Al-Tamimi, Lewis & Abu-Khalaf (Automatica 43(3):473–481, 2007) solve the Game ARE (GARE) model-free and validate on an F-16 autopilot. ScienceDirect

8. The HJB is a PDE → physics-informed Bellman residual. Your PINN machinery can directly minimize the HJB residual as an extra loss (Wang & Wu, AIChE J. 2024; Meng et al. 2024).

9. The PITNN is a learned world model → TD-MPC2-style latent planning is a natural extension of your multi-step prediction loss.

10. Keep $V=e^T Pe$ classical as the backbone; add a learned neural residual (generalized Lyapunov, Long et al. NeurIPS 2025) only if you need larger certified regions.

Details

Connection 1 — MRAS Lyapunov function IS a value function (ADP / Integral RL)

The identity. For LQR with $V(x)=\tfrac{1}{2}x^T Px$ and cost $\tfrac{1}{2}\int(x^T Qx+u^T Ru)d\tau$, substituting a feedback $u=-Kx$ into the Bellman equation and requiring it hold for all trajectories gives the **Lyapunov equation** (Kleinman policy-evaluation step):

$(A - BK)^T P + P(A - BK) + Q + K^T R K = 0$

with policy improvement $K_{j+1} = R^{-1}B^T P_j$. At convergence $K_j = K_{j+1}$, substitution recovers the **CARE**: $A^T P + PA + Q - PBR^{-1}B^T P = 0$. arXiv Lewis's RL short-course states verbatim that "for the LQR case Policy Iteration … interleaves [the Lyapunov equation] and [the gain update], … which was first delivered by David Kleinman" (the iterative-Riccati technique was published in Kleinman 1968, DOI 10.1109/TAC.1968.1098829). SCIRP For your MRAS error system $\dot{e} = A_m e$ + (perturbation), the bare Lyapunov equation $A_m^T P + PA_m = -Q$ is the **R→∞ / B=0 degenerate case** (pure policy evaluation, no control penalty term). So your P matrix is already a value-function Hessian; you are doing one step of policy evaluation and never iterating.

Vrabie–Lewis Integral RL Bellman equation (DOI 10.1016/j.neunet.2009.03.008). The key move that makes this model-free is the IRL Bellman (difference) equation:

$V(x(t)) = \int_{t}^{t+T} r(x,u)\, d\tau + V(x(t+T))$

with policy evaluation $V^j(x(t)) = \int_{t}^{t+T} d\tau + V^j(x(t+T))$ and policy improvement $u^{j+1} = -R^{-1}B^T \nabla V^j$. The crucial property (Vrabie–Lewis Lemma): this value equals the solution of the Bellman PDE, yet the IRL Bellman equation **does not contain A** (the internal/drift dynamics), so policy evaluation is done from measured data along trajectories — exactly the "partially unknown" setting PITS-MRAS targets. With value-function approximation $V(x) = W^T \phi(x)$, this becomes the linear-in-weights equation $W^T[\phi(x(t)) - \phi(x(t+T))] = \int_{t}^{t+T} r\, d\tau$, solved online by recursive least squares under persistence of excitation.

Recast of your adaptation law as an integral-RL update. Your current law is $d\theta/dt = -\Gamma[\nabla L_{total} + \beta_{MRAS} \cdot e \cdot \phi_\theta]$. Add a **critic** $\hat{W}_c$ parameterizing $\hat{V}(e) = \hat{W}_c^T \phi_c(e)$ and a temporal/integral-RL residual (the "IRL Bellman error"): $\delta_{IRL}(t) = \int_{t-T}^{t} (e^T Q e + u^T R u)\, ds + \hat{V}(e(t)) - \hat{V}(e(t-T))$, and update the critic by least-squares/gradient on $\tfrac{1}{2}\delta_{IRL}^2$. The actor (your feedback gain K_{fb}) then improves via $K_{fb} \leftarrow R^{-1}B^T \hat{P}$ where $\hat{P}$ is read off the critic. This is exactly Vamvoudakis & Lewis (2010) "synchronous policy iteration" (Automatica 46(5):878–888, DOI 10.1016/j.automatica.2010.02.018), which adapts actor and critic *simultaneously* in continuous time with a persistence-of-excitation guarantee and a nonstandard extra actor term for closed-loop stability — structurally identical to your $\beta_{MRAS} \cdot e \cdot \phi_\theta$ term.

New components for #1: (i) a critic head $\hat{V}(e) = \hat{W}_c^T \phi_c(e)$; (ii) an integral-reinforcement accumulator buffering $\int r\, ds$ over a window T; (iii) loss term $L_{IRL} = \tfrac{1}{2}\|\delta_{IRL}\|^2$; (iv) replace the static K_{fb} with the policy-improvement update. Relevant prior art: the MR-ARL framework (arXiv:2402.14483, "Model Reference Adaptive Reinforcement Learning for Robustly Stable On-Policy Data-Driven LQR") explicitly fuses MRAC with RL using a variable reference model containing the current value function arXiv and provides robust-stability certificates — the closest published analog to PITS-MRAS, and should be cited.

Connection 2 — Port-Hamiltonian ↔ passivity-based RL / energy shaping

A port-Hamiltonian system $\dot{x} = (J-R)\nabla H + g\cdot u$, $y = g^\top \nabla H$ satisfies the **dissipation inequality** $\dot{H} = -\nabla H^\top R \nabla H + y^\top u \le y^\top u$, i.e. H is a *storage function*. This is the same inequality as $\dot{V} \le$ (supply) in L2-gain/H∞ analysis; passivity with a positive storage function is the energy-domain statement of a value function. In IDA-PBC (Ortega et al.), control shapes the closed loop into a target PH system with desired Hamiltonian H_d arXiv whose minimum is the setpoint — H_d is the closed-loop Lyapunov/value function, and solving the matching PDEs is the energy-domain analog of solving HJB.

Key papers: **Duong & Atanasov**, "Port-Hamiltonian Neural ODE Networks on Lie Groups for Robot Dynamics Learning and Control" (arXiv:2401.09520; RSS 2021 precursor "Hamiltonian-based Neural ODE Networks on the SE(3) Manifold") — learns H, J, R, g with neural nets and derives energy-shaping + damping-injection control, arxiv directly parallel to your decoder. **Zhong, Dey & Chakraborty** (Symplectic/Dissipative ODE-Nets, 2020) handle the dissipative R term. Robotics: Science and Systems **Sprangers, Lopes & Babuška**, "Reinforcement learning for port-Hamiltonian systems" / energy-balancing PBC (arXiv:1212.5524) parameterize EB-PBC and tune it with actor-critic RL arxiv — the explicit bridge you want. **Sanchez-Escalonilla, Reyes-Báez & Jayawardhana**, "Total Energy Shaping with Neural Interconnection and Damping Assignment" (L4DC 2022, PMLR v168) solve the IDA-PBC matching PDEs as a supervised-learning problem with explicit Lyapunov-stability conditions. Proceedings of Machine Learning R

Architectural recommendation: treat H_θ as a CLF candidate directly: enforce $H_\theta(q^*,p^*)=0$, $\nabla H_\theta(q^*,p^*)=0$, and $\nabla^2 H_\theta > 0$ at the target (equilibrium-assignment + positive-definiteness losses), and use the damping-injection control $u = -g^+(\nabla H \cdot K_d)$ as your u_{aux}. Add an energy-shaping loss matching ∇H_d. The L2-gain bound γ becomes a tunable hyperparameter linking this to Connection 7.

Connection 3 — Costate = critic gradient = ∇H

From PMP, the costate satisfies $\lambda(t) = \partial V/\partial x$ (e.g., Fleming–Rishel Thm I.6.2); the optimal control is $u^* = -R^{-1}g^\top \lambda = -R^{-1}g^\top \nabla V$. In the port-Hamiltonian decoder, $\nabla_q H$ and $\nabla_p H$ are the generalized force and velocity. So **three objects coincide**: PMP costate λ, critic gradient $\nabla \hat{V}$, and (under the value=energy identification) ∇H. Recent work makes this architecturally explicit: the "Neural Hamiltonian Operator" (arXiv:2507.01313) parameterizes the FBSDE adjoint and the value-gradient ansatz with networks and trains them to satisfy PMP consistency; arxiv PG-DPO (arXiv:2412.13101) tracks a policy-fixed BSDE for the adjoint λ_t and adds an *alignment penalty* nudging the policy toward the Pontryagin solution; arXiv and "End-to-end Training of High-Dimensional Optimal Control with Implicit Hamiltonians" (arXiv:2510.00359) exploits "the gradient of the value function yields the adjoint variable" to build feedback laws. arXiv

How to make c_t approximate λ(t). Add a *costate head* that maps the attention context to a costate estimate $\hat{\lambda}_t = W_\lambda c_t$, and enforce two consistency losses: (a) the **gradient-consistency loss** $L_{costate} = \|\hat{\lambda}_t - \nabla_e \hat{V}(e_t)\|^2$ (enforces $\lambda=\nabla V$), and (b) the **adjoint-dynamics residual** $-\dot{\lambda} = \partial H/\partial x$ evaluated along trajectories (enforces the PMP backward ODE). Then define the control head as $u = -R^{-1}\hat{g}^\top \hat{\lambda}_t$. Architecturally, "enforcing $\lambda=\nabla V$ in a network" means making the action head the analytic gradient of the value head (a single scalar value network differentiated by autodiff) rather than an independent actor — this guarantees the identity by construction (the same trick PINNs use to compute exact derivatives, and what HJBPPO does by treating the value net as a PINN).

Connection 4 — MRAS adaptation law ↔ deterministic policy gradient

The deterministic policy gradient theorem (Silver, Lever, Heess, Degris, Wierstra & Riedmiller, "Deterministic Policy Gradient Algorithms," PMLR 32(1):387–395, ICML 2014) states $\nabla_\theta J = E[\nabla_\theta \mu_\theta(s) \cdot \nabla_a Q(s,a)|_{a=\mu_\theta(s)}]$ — the policy Jacobian chained against the action-gradient of the critic; in the authors' words, "the deterministic policy gradient has a particularly appealing form: it is the expected gradient of the action-value function." ACM Digital Library Proceedings of Machine Learning R Your MRAS update $\dot{\theta}=-\Gamma\, e\, \phi$ is a gradient step on $V(e,\tilde{\theta})$ where $\phi_c=[e^\top, r^\top, x_p^\top]^\top$ is the regressor. The formal correspondence: ϕ_c plays the role of $\nabla_\theta \mu_\theta$ (sensitivity of the control/output to parameters), and e (the tracking error, $\propto \nabla_e V$ via P) plays the role of $\nabla_a Q$. So the MRAS law **is** a deterministic actor update with the critic-gradient replaced by the linear surrogate Pe. Your hybrid gradient+MRAS update is therefore precisely an actor-critic algorithm where the "critic" is currently fixed (the static P) rather than learned. Making the critic learnable (Connection 1) upgrades MRAS → full actor-critic. Caveat: under function approximation, DPG requires a *compatible* critic for the gradient to be unbiased (Silver Thm 3); see "Compatible Gradient Approximations for Actor-Critic Algorithms" (arXiv:2409.01477) for the failure mode and a zeroth-order fix. arxiv

New term: replace the surrogate e in your actor update with the learned critic gradient: $\dot{\theta} = -\Gamma\, \phi_c \cdot (\nabla_a \hat{Q})$ where $\nabla_a \hat{Q}$ reduces to Pe in the LQR limit but generalizes to nonlinear costs.

Connection 5 — SAC entropy ↔ control-penalty R-matrix

SAC (Haarnoja, Zhou, Abbeel & Levine, "Soft Actor-Critic: Off-Policy Maximum Entropy Deep RL with a Stochastic Actor," PMLR 80:1861–1870, ICML 2018, arXiv:1801.01290) maximizes $J(\pi)=\Sigma\, E[r + \alpha \cdot H(\pi(\cdot|s))]$. arxiv For the **max-entropy LQR/LQG** case, the optimal soft policy is Gaussian $\pi(u|x)=N(-Kx, \Sigma)$ with covariance $\Sigma \propto \alpha \cdot R^{-1}$: the entropy temperature α scales inversely with the control-penalty weight. Intuitively, a large R (expensive control) ↔ small-variance, conservative policy; large α (high entropy bonus) ↔ exploratory policy. Your penalty $L_{control}=E[\|u\|^2+\lambda_{\Delta u}\|\Delta u\|^2]$ is the negative-reward form of

the LQR control cost; reformulating as a max-entropy objective means **adding −α H(π) as the control regularizer**, i.e. $J = E[\Sigma(-e^T Q e - u^T R u) + \alpha H(\pi)]$. Recent confirmation that SAC recovers the max-entropy LQR optimum: arXiv:2512.23870 ("Max-Entropy RL with Flow Matching and A Case Study on LQR"). arXiv

Can SAC be the PITS-MRAS controller? Yes — make the actor output a Gaussian over u with mean = $K_{fb} e + K_{ff} r + u_{aux}$ and learned covariance; identify α with your control weight (use SAC's automatic temperature tuning to set it against a target-entropy that encodes your desired R). The critic $\hat{Q}$ is shared with Connection 1's value head. The Δu penalty maps to an action-rate term, easily added to the reward. This gives you exploration + robustness — Eysenbach & Levine ("Maximum Entropy RL (Provably) Solves Some Robust RL Problems," arXiv:2103.06257, ICLR 2022) prove theoretically that MaxEnt RL maximizes a lower bound on a robust-RL objective — while preserving the LQR/MRAS structure as the policy mean.

Connection 6 — CBF-QP safety filter

Replace the informal `V̇<0 → apply; else backup` with the CLF-CBF-QP of Ames et al. ("Control Barrier Function Based Quadratic Programs for Safety Critical Systems," arXiv:1609.06408 / IEEE TAC 2017; ACC 2015 unified CLF-CBF-QP at ames.caltech.edu/CLF_QP_ACC_final.pdf). For an affine system $\dot{x}=f+gu$, a CBF $h(x)$ ($h>0$ = safe) enforces forward invariance via $L_f h + L_g h \cdot u \geq -\gamma h(x)$. The **safety filter** is: $u_{safe} = \arg\min_u \|u - u_{RL}\|^2$ s.t. $L_f h + L_g h \cdot u \geq -\gamma h(x)$.

Closed-form single-constraint solution. Let $a = L_f h(x) + L_g h(x) \cdot u_{RL} + \gamma h(x)$ and $b = (L_g h(x))^T$. If $a \geq 0$ the nominal control is already safe ($u_{safe}=u_{RL}$); else $u_{safe} = u_{RL} + (-a / \|b\|^2) \cdot b = u_{RL} - [(L_f h + L_g h \cdot u_{RL} + \gamma h)/\|L_g h\|^2] \cdot (L_g h)^T$. This is the minimal-norm projection onto the safe half-space — extremely cheap, no QP solver needed for one constraint.

Can V(e) serve as the CBF? They share quadratic structure but encode opposite things: a CLF must *decrease* ($\dot{V}<0$) while a CBF must stay *positive* (forward invariance of $\{h\geq 0\}$). You can derive a CBF from your performance set, e.g. $h(e) = c - e^T P e$ (safe = tracking error inside an ellipsoid), in which case $L_f h = -2e^T P A_m e$ and $L_g h = -2e^T P B$. This makes the CBF and CLF *the same quadratic form* with flipped sign and an offset c — and means one matrix P serves both stability (CLF) and safety (CBF) roles. For relative-degree-2 constraints use High-Order CBFs (Xiao, Belta & Cassandras, "Adaptive Control Barrier Functions," arXiv:2002.04577). arxiv

Connection 7 — H∞ / two-player zero-sum game for robust PITS-MRAS

Al-Tamimi, Lewis & Abu-Khalaf (2007), Automatica 43(3):473–481 (DOI 10.1016/j.automatica.2006.09.019) formulate robust control as the zero-sum game

ACM Digital Library with value $V^*(x_k) = \min_u \max_w \Sigma[x_i^T R x_i + u_i^T u_i - \gamma^2 w_i^T w_i]$, solved by the **Game Algebraic Riccati Equation (GARE)**: $P = A^T P A + R - [A^T P B \; A^T P E] \cdot [I + B^T P B \; B^T P E; \; E^T P B \; E^T P E - \gamma^2 I]^{-1} \cdot [B^T P A; \; E^T P A]$. Crucially, the paper states verbatim that "the GARE in Eq. (5) will be the algebraic Riccati equation (ARE) if $E = 0$" derongliu — so your MRAS Lyapunov/CARE is the **disturbance-free (E=0, γ→∞) special case** of the GARE. The saddle-point requires $I - \gamma^{-2} E^T P E > 0$, derongliu exactly the L2-gain bound γ that the reference model implicitly specifies. They solve it model-free via a Q-function $Q(x,u,w) = [x^T \; u^T \; w^T] H [x^T \; u^T \; w^T]^T$ whose kernel H contains all the game data, ScienceDirect with control gain $L = (H_{uu} - H_{uw} H_{ww}^{-1} H_{wu})^{-1} (H_{uw} H_{ww}^{-1} H_{wx} - H_{ux})$ derongliu and the dual disturbance gain K. **Validation:** a discretized F-16 short-period autopilot, states $x = [\alpha, q, \delta e]$ (angle of attack, pitch rate, elevator deflection), sampling $T = 0.1$ s, $\gamma = 1$, derongliu with A,B,E given explicitly and GARE solution $P = [[15.51, 12.41, -0.009], [12.41, 15.60, -0.008], [-0.009, -0.008, 1.01]]$; derongliu the model-free critic converges to this P and the actor/adversary networks to the Nash gains. ACM Digital Library Continuous-time analogs: Vrabie & Lewis (2011) IRL for ZS games; Vamvoudakis & Lewis (2012, Int. J. Robust Nonlinear Control 22(13), DOI 10.1002/rnc.1760) online synchronous PI with an **actor/critic/disturbance** three-network structure. Wiley Online Library

Architectural change: add a *disturbance/adversary head* $\hat{w} = K_w \cdot x_p$ that maximizes the cost, train the critic on the game-Bellman residual, and set γ from your reference-model performance bound. This directly hardens PITS-MRAS against the plant nonlinearities and parameter drift your physics decoder cannot capture.

Connection 8 — Physics-informed Bellman / HJB residual

The HJB is a PDE: $0 = \min_u [\ell(x,u) + \nabla_x V \cdot f(x,u)]$, and at the optimum $0 = \ell(x,u^*) + \nabla_x V \cdot f(x,u^*)$ with $u^* = -R^{-1} g^T \nabla_x V$. Since your PINN already enforces PDE residuals, add the **HJB residual loss**: $L_{HJB} = \| \ell(x,u^*) + \nabla_x \hat{V} \cdot \hat{f}_\theta(x,u^*) \|^2$, $u^* = -\tfrac{1}{2} R^{-1} \hat{g}^T \nabla_x \hat{V}$, evaluated by autodiff on the value head. This is exactly the approach of **Wang & Wu, "Physics-informed reinforcement learning for optimal control of nonlinear systems," AIChE J. 70(10):e18542, 2024 (DOI 10.1002/aic.18542)** — two PINNs (value + policy) encoding Lyapunov-stability and policy-iteration convergence conditions, model-free, ResearchGate with closed-loop stability proofs. Wiley Online Library Also: **Meng et al. 2024 "Physics-Informed Neural Network Policy Iteration"** (ELM-PI / PINN-PI, proven uniform convergence to the HJB viscosity solution); Amartyamukherjee **HJBPPO** (arXiv:2302.00237) which treats the PPO value net as a PINN and adds an MSE-HJB term; OpenReview **Mukherjee & Liu** (ICML 2023 workshop) actor-critic-as-PINN for a 1D PDE battery-cooling model; Amartyamukherjee and **Wallace & Si (JMLR 2024)** physics-informed continuous-time RL with performance guarantees. The discounted/entropy-regularized HJB variant is in arXiv:2508.01720. arxiv Source caveat: HJBPPO found the HJB loss did **not** consistently improve over standard PPO across MuJoCo (improved in only some environments),

ResearchGate so treat L_HJB as a regularizer whose weight needs tuning, not a guaranteed win.

New terms: L_HJB above; optionally a Lyapunov-decrease penalty L_dec = ReLU(∇V̂·f̂ + ℓ) to enforce V̇≤−ℓ.

Connection 9 — World-model / latent-space MPC (TD-MPC2)

Your PITNN (ẋ_p ≈ f̂_θ) is a learned world model; TD-MPC2 (Hansen, Su, Wang, ICLR 2024, arXiv:2310.16828) performs local trajectory optimization (MPPI) in the latent space of a learned implicit model and bootstraps beyond the planning horizon arXiv with a learned terminal value function trained by TD learning. DreamerV3 (Hafner, Pasukonis, Ba & Lillicrap, "Mastering diverse control tasks through world models," *Nature* 640(8059):647–653, 2 Apr 2025, DOI 10.1038/s41586-025-08744-2; preprint arXiv:2301.04104) instead trains an actor-critic purely on imagined latent rollouts from an RSSM. arXiv

How to structure your latent space for TD-MPC2-style planning: (i) make the LSTM-encoder context c_t the *latent state* z_t and learn a latent dynamics head z_{t+1}=d_θ(z_t,u_t) plus a reward/cost head and a terminal value head V̂(z); (ii) at inference, run MPPI/CEM over u-sequences using d_θ rollouts plus terminal V̂; (iii) **the key identity**: your multi-step prediction loss Σ‖x̂_{t+k}−x_{t+k}‖² and TD-MPC2's *latent consistency* loss are the same self-supervised objective, while TD-MPC2 adds a **temporal-difference loss** ‖Q̂(z_t,u_t) − (r_t + γ Q̂(z_{t+1},u_{t+1}))‖² that your framework currently lacks — adding it turns your predictive model into a value-equipped planner. Train dynamics, reward, and value jointly (TD-MPC2 does it in one TD loop). Caveat: TD-MPC2's value/latent is task-specific; your port-Hamiltonian latent gives a physics-grounded, reusable z that should improve cross-task transfer relative to TD-MPC2's task-oriented latent.

Connection 10 — Neural Lyapunov certificates

Chang, Roohi & Gao, "Neural Lyapunov Control," NeurIPS 2019 (arXiv:2005.00611) jointly learn a controller and a neural Lyapunov function via a *learner-falsifier* (CEGIS) loop: ACM Digital Library arxiv the learner minimizes the **Lyapunov risk** E[ReLU(−V(x)) + ReLU(V̇(x)+ε‖x‖²)] + V(0)², and a δ-complete SMT solver (dReal) searches for counterexamples (states violating V>0 or V̇<0), which are appended to the training set; NeurIPS termination with no counterexample yields a *formal* certificate over the domain, with much larger regions of attraction than LQR/SOS. ACM Digital Library Long et al., "Certifying Stability of RL Policies using Generalized Lyapunov Functions," NeurIPS 2025 (arXiv:2505.10947) relax the strict step-wise decrease to a **multi-step / average decrease** arxiv and certify by augmenting the RL value function with a learned neural residual V_gen = V_RL + ϕ(x) + step-weighting; for LQR they show the augmented value satisfies LMI-certifiable generalized-Lyapunov conditions even under discounting (building on Postoyan

et al. 2017). arxiv

Recommendation for PITS-MRAS: *keep* the classical $V(e)=e^{\top}Pe$ as the backbone — for the linear reference model it is exact, gives you the CARE/Kleinman/Connection-1 machinery for free, and is trivially verifiable. *Add* a learned residual $V = e^{\top}Pe + \phi\_NN(e)$ (generalized-Lyapunov style) **only** for the nonlinear plant regime where the quadratic form is too conservative, and certify with the multi-step condition + a falsifier. This hybrid keeps analytic guarantees where they hold and learns flexibility where needed. The tradeoff: neural Lyapunov + SMT verification is expensive and scales poorly with state dimension (Long et al. report longer verification times for the multi-step bounds), so reserve it for final certification, not the inner training loop.

Recommendations (prioritized by rigor-per-unit-effort)

Tier 1 — do first (high rigor, low complexity):

1. **Integral-RL critic + synchronous PI (Connection 1).** Add the critic head $\hat{V}(e)$, the integral-reinforcement accumulator, L_IRL, and the $K\_fb \leftarrow R^{-1}B^{\top}\hat{P}$ policy-improvement step. This upgrades MRAS from one-shot policy evaluation to full policy iteration and gives model-free optimality. Cite Vrabie–Lewis 2009 and Vamvoudakis–Lewis 2010 as the formal backbone; MR-ARL (arXiv:2402.14483) as the MRAC-RL bridge. *Benchmark to change the plan:* if persistence of excitation cannot be guaranteed online, fall back to off-policy/batch least-squares on a replay buffer.
2. **CLF-CBF-QP safety filter (Connection 6).** Replace the heuristic with the closed-form single-constraint projection; derive $h(e)=c-e^{\top}Pe$ so one P serves CLF and CBF. Near-zero compute, immediate formal forward-invariance.
3. **Costate head with $\lambda=\nabla V$ by construction (Connection 3).** Make the action head the autodiff gradient of a scalar value head; add L_costate and the adjoint-dynamics residual. Small architectural change that buys you PMP-consistency and interpretability.

Tier 2 — do next (high payoff, moderate complexity): 4. **HJB physics-informed residual (Connection 8).** Add L_HJB to your existing physics loss; tune its weight (HJBPPO caveat). Aligns directly with your PINN infrastructure. 5. **Port-Hamiltonian H as CLF + energy-shaping control (Connection 2).** Equilibrium-assignment, positive-definiteness, and energy-shaping losses on H_θ; use damping injection as u_aux. 6. **SAC as the controller with $\alpha \equiv R$ (Connection 5).** Gaussian actor with MRAS-structured mean; automatic temperature tuning to encode R. Adds exploration + robustness.

Tier 3 — do if robustness / certification demands it (high complexity): 7. **H∞ two-player game head (Connection 7).** Add the disturbance/adversary network and game-

Bellman critic; set γ from the reference model. Use when plant uncertainty is large. 8. **TD-MPC2 latent planning (Connection 9).** Add latent dynamics/reward/value heads and the TD loss; enables receding-horizon latent planning. Biggest engineering lift. 9. **Generalized neural Lyapunov certificate (Connection 10).** $V=e^{\top}Pe+\phi\_NN(e)$ with multi-step decrease + falsifier, for final certification of the nonlinear regime only.

Threshold guidance: If your plant stays close to the linear reference model (small tracking error, slow parameter drift), Tier 1 alone suffices and everything has classical guarantees. If the physics decoder shows large residuals (strong nonlinearity), escalate to Tier 2 (HJB + energy shaping). If you face adversarial disturbances or must certify a region of attraction for deployment, add Tier 3.

Caveats

- **PE / exploration bias:** every IRL/ADP method (Connections 1, 7) needs persistence of excitation; injecting probing noise biases estimates unless handled (off-policy IRL or the bias-correction in Kiumarsi et al. 2017).
- **Compatibility (Connection 4):** the DPG identity is unbiased only with a *compatible* critic; a naive learned critic can give a wrong policy gradient.
- **HJB-loss is not a guaranteed win (Connection 8):** HJBPPO reported no consistent improvement over PPO on MuJoCo; treat L_HJB as a tunable regularizer.
- **Verification cost (Connection 10):** SMT/falsifier certification scales poorly with dimension; keep the quadratic backbone and use neural residuals sparingly.
- **The ½ convention:** Lewis's short-course carries a ½ in the cost/value ($V=\tfrac{1}{2}x^{\top}Px$); the Vrabie–Lewis journal article often omits it. Keep your factor-of-2 bookkeeping consistent when you wire $R^{-1}B^{\top}P$ into the gain.
- **Discounting vs. Lyapunov (Connection 10):** discounted RL value functions can fail the classical step-wise Lyapunov decrease; the generalized (multi-step) condition or Postoyan's quadratic-residual augmentation is needed.
- **Date/attribution note:** Kleinman's iterative-Riccati technique is dated 1968 (IEEE TAC AC-13(1):114–115); some secondary sources informally cite "1963" for the original idea — use 1968 for the published reference.

Key references with identifiers

- Kleinman, *On an Iterative Technique for Riccati Equation Computations*, IEEE TAC AC-13(1):114–115, 1968 — DOI 10.1109/TAC.1968.1098829. SCIRP

- Vrabie & Lewis, *Neural network approach to CT direct adaptive optimal control…*, Neural Networks 22(3):237–246, 2009 — DOI 10.1016/j.neunet.2009.03.008. PubMed
- Vamvoudakis & Lewis, *Online actor–critic algorithm…*, Automatica 46(5):878–888, 2010 — DOI 10.1016/j.automatica.2010.02.018.
- Al-Tamimi, Lewis & Abu-Khalaf, *Model-free Q-learning designs for linear DT zero-sum games…H∞*, Automatica 43(3):473–481, 2007 — DOI 10.1016/j.automatica.2006.09.019. ScienceDirect
- Vamvoudakis & Lewis, *Online solution of nonlinear two-player zero-sum games…*, Int. J. Robust Nonlinear Control 22(13), 2012 — DOI 10.1002/rnc.1760.
- Ames, Xu, Grizzle & Tabuada, *Control Barrier Function Based QPs for Safety Critical Systems*, IEEE TAC 2017 — arXiv:1609.06408.
- Xiao, Belta & Cassandras, *Adaptive Control Barrier Functions* — arXiv:2002.04577.
- Silver et al., *Deterministic Policy Gradient Algorithms*, ICML 2014 — PMLR 32(1):387–395. dblp
- Haarnoja, Zhou, Abbeel & Levine, *Soft Actor-Critic*, ICML 2018 — PMLR 80:1861–1870 / arXiv:1801.01290.
- Eysenbach & Levine, *Maximum Entropy RL (Provably) Solves Some Robust RL Problems*, ICLR 2022 — arXiv:2103.06257. arXiv
- Duong, Altawaitan, Stanley & Atanasov, *Port-Hamiltonian Neural ODE Networks on Lie Groups…* — arXiv:2401.09520.
- Sprangers, Lopes & Babuška, *Reinforcement learning for port-Hamiltonian systems (EB-PBC)* — arXiv:1212.5524.
- Sanchez-Escalonilla, Reyes-Báez & Jayawardhana, *Total Energy Shaping with Neural IDA-PBC*, L4DC 2022 — PMLR v168 / arXiv:2112.12999.
- Wang & Wu, *Physics-informed RL for optimal control of nonlinear systems*, AIChE J. 70(10):e18542, 2024 — DOI 10.1002/aic.18542.
- Meng et al., *Physics-Informed Neural Network Policy Iteration* (ELM-PI / PINN-PI), 2024.
- Mukherjee & Liu, *Bridging PINNs with RL: HJBPPO* — arXiv:2302.00237.
- Hansen, Su & Wang, *TD-MPC2: Scalable, Robust World Models for Continuous Control*, ICLR 2024 — arXiv:2310.16828.
- Hafner, Pasukonis, Ba & Lillicrap, *Mastering diverse control tasks through world models*, Nature 640(8059):647–653, 2025 — DOI 10.1038/s41586-025-08744-2. Nature

- Chang, Roohi & Gao, *Neural Lyapunov Control*, NeurIPS 2019 — arXiv:2005.00611.
- Long et al., *Certifying Stability of RL Policies using Generalized Lyapunov Functions*, NeurIPS 2025 — arXiv:2505.10947.
- *MR-ARL: Model Reference Adaptive Reinforcement Learning…* — arXiv:2402.14483.
- *Neural Hamiltonian Operator* — arXiv:2507.01313; *PG-DPO* — arXiv:2412.13101; *Implicit-Hamiltonian optimal control* — arXiv:2510.00359.